

DEEP LEARNING – HANDWRITTEN EXAM NOTES

UNIT 2: Multi-Layer Perceptron, Backpropagation, Loss Functions, Regularization

B.Tech Mid-1 Examination Preparation | Complete Question Bank Solutions

PART A – SHORT ANSWER QUESTIONS

Question 1

Define multi-layer perceptron (MLP)

Definition

A Multi-Layer Perceptron (MLP) is a class of feedforward artificial neural network consisting of at least three layers of nodes: an **input layer**, one or more **non-linear hidden layers**, and an **output layer**.

Key Characteristics

- **Fully Connected (Dense):** Every node in one layer connects to every node in the following layer.
- **Non-Linear Activation:** Hidden nodes use non-linear activation functions (e.g., ReLU, Sigmoid) to approximate complex non-linear mappings.
- **Universal Approximation:** An MLP with a single hidden layer can approximate any continuous function (Cybenko Theorem).
- **Training:** Trained using the **Backpropagation algorithm** with Gradient Descent.

Question 2

What are the three variants of Gradient Descent

Variant	Data Used per Update	Key Advantage	Disadvantage
1. Batch Gradient Descent (BGD)	Entire training dataset (all N samples)	Smooth, stable convergence to minimum	Very slow for large datasets; high memory usage
2. Stochastic GD (SGD)	Exactly 1 sample per update	Fast updates; can escape shallow local minima	Noisy, highly oscillatory loss curve
3. Mini-Batch GD	Small subset (e.g., 32, 64, 128)	Best trade-off; harnesses GPU	Requires tuning batch size hyperparameter

samples)

vectorization

Question 3

List out any three activation functions used in hidden layer

1. 1. Rectified Linear Unit (ReLU):

$f(z) = \max(0, z)$. Outputs z if $z > 0$, else 0 . Range: $[0, \infty)$. Default choice for deep hidden layers because it avoids vanishing gradients for $z > 0$ and computes extremely fast.

2. 2. Hyperbolic Tangent (tanh):

$f(z) = (e^z - e^{-z}) / (e^z + e^{-z})$. Range: $(-1, +1)$. Zero-centered activation that makes gradient descent updates more balanced than sigmoid.

3. 3. Leaky ReLU:

$f(z) = \max(\alpha z, z)$ (with $\alpha \approx 0.01$). Prevents the "Dying ReLU" problem by allowing a small, non-zero gradient when $z < 0$.

Question 4

What are various types of Loss functions and their suitable applications

Loss Function	Mathematical Formula	Suitable Application
Mean Squared Error (MSE)	$L = (1/n) \sum (y_i - \hat{y}_i)^2$	Regression (House price prediction, stock forecasting)
Mean Absolute Error (MAE / L1)	$L = (1/n) \sum y_i - \hat{y}_i $	Regression with heavy outliers (robust training)
Binary Cross-Entropy (BCE)	$L = - [y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$	Binary Classification (Spam detection, Disease yes/no)
Categorical Cross-Entropy	$L = - \sum y_c \log(\hat{y}_c)$	Multi-Class Classification (ImageNet, MNIST digits)

Question 5

Define overfitting and underfitting

Concept	Definition	Train vs Test Error	Remedies
Overfitting (High Variance)	Model memorizes training data and noise, failing to generalize to new unseen data.	Very Low Train Error, High Test Error	Dropout, L2 Weight Decay, Early Stopping, Data Augmentation

Underfitting (High Bias)	Model is too simple to capture the underlying patterns in the data.	High Train Error, High Test Error	Increase model depth/capacity, add features, train longer
-----------------------------	---	-----------------------------------	---

Question 6

Give the list of types of Regularization methods

1. **L2 Parameter Regularization (Ridge / Weight Decay):** Adds $(\lambda/2) \sum w^2$ to penalize large weights.
2. **L1 Parameter Regularization (Lasso):** Adds $\lambda \sum |w|$ to produce sparse weight matrices.
3. **Dropout:** Randomly deactivates a probability p of neurons during each training forward pass.
4. **Data Augmentation:** Artificially expands training data using rotations, crops, and flips.
5. **Early Stopping:** Halts training when validation loss starts increasing.
6. **Batch Normalization & DropConnect:** Regularizes activations and synaptic weights.

----- PART B – ESSAY / 10-MARK QUESTIONS -----

Essay Question 1

Explain about back propagation algorithm for training a neural network

1. Introduction

Backpropagation (Backward Propagation of Errors) is the core supervised learning algorithm used to train multi-layer artificial neural networks. It efficiently computes the gradient of the loss function with respect to every weight and bias in the network using the **Calculus Chain Rule**.

2. Two-Phase Mechanism

Phase 1: Forward Pass

Input vector x is propagated forward layer by layer until it produces output $\hat{y}$:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Phase 2: Backward Pass (Error Gradient Propagation)

The loss $E = (1/2)\sum (y - \hat{y})^2$ is computed at the output layer. Using the chain rule, gradients flow backward:

Chain Rule: $\partial E / \partial w_{ij} = (\partial E / \partial a_j) \cdot (\partial a_j / \partial z_j) \cdot (\partial z_j / \partial w_{ij})$

Error Signal (δ): $\delta_j = (\partial E / \partial z_j)$

- **Output Layer:** $\delta_k = (\hat{y}_k - y_k) \cdot f'(z_k)$
- **Hidden Layer:** $\delta_j = (\sum \delta_k w_{jk}) \cdot f'(z_j)$

Weight Update Equation

$$w_{ij}(t+1) = w_{ij}(t) - \eta \cdot \delta_j \cdot a_i$$

Conclusion: Backpropagation transformed AI by enabling arbitrarily deep architectures to learn hierarchical representations directly from data.

Essay Question 2

What is a feed forward neural network? Explain gradient based learning in a feedforward network

1. Feedforward Neural Network (FFNN)

A Feedforward Neural Network is an artificial neural network where connections between nodes do not form a cycle or loop. Information moves in only one direction — forward from the input nodes, through the hidden nodes, to the output nodes.

2. Gradient-Based Learning Framework

1. **Cost Function Formulation:** Define a loss function $J(\theta)$ quantifying discrepancy between predictions and true targets.
2. **Gradient Computation:** Calculate the vector of partial derivatives $\nabla_{\theta} J(\theta)$ using backpropagation.
3. **Parameter Update:** Move parameters in the direction of steepest descent:

$$\theta(t+1) = \theta(t) - \eta \nabla_{\theta} J(\theta)$$

Essay Question 3

Explain the difficulties in training deep neural networks

1. 1. Vanishing Gradient Problem:

During backpropagation, gradients are multiplied layer-by-layer. Saturating activations (Sigmoid, tanh) have derivatives ≤ 0.25 . Multiplying many small fractions causes gradients to shrink exponentially towards zero, leaving early layers untrained.

Remedies: ReLU activation, He initialization, Residual connections (ResNet).

2. Exploding Gradient Problem:

If weights are initialized too large, repeated matrix multiplications cause gradients to grow exponentially, causing unstable updates and NaN overflows.

Remedies: Gradient Clipping, Xavier/Glorot initialization, Batch Normalization.

3. Saddle Points & Plateaus:

In high-dimensional non-convex loss surfaces, saddle points (where gradient is zero but not a minimum) are far more common than local minima, slowing down standard SGD.

Remedies: Adaptive optimizers (Adam, RMSProp), Momentum.

4. Internal Covariate Shift:

As parameters of earlier layers change, the input distribution to deeper layers shifts continuously, slowing convergence.

Remedies: Batch Normalization, Layer Normalization.

Essay Question 4

Explain various types of regularization and how does Regularization helps in reducing overfitting

1. How Regularization Reduces Overfitting

Regularization introduces a **penalty term** or stochastic constraint to prevent weights from growing unboundedly large. Smaller weights produce smoother decision boundaries, making the model robust to noise and highly generalizable.

2. Regularization Methods in Detail

1. L2 Regularization (Weight Decay):

$$J_{\text{regularized}} = J_0 + (\lambda / 2n) \sum w^2$$

Weight update: $w \leftarrow w(1 - \eta\lambda/n) - \eta(\partial J_0 / \partial w)$. Drives weights towards small values without forcing them to exact zero.

2. L1 Regularization (Lasso):

$$J_{\text{regularized}} = J_0 + (\lambda / n) \sum |w|$$

Forces irrelevant weights to become exactly zero, acting as an automatic feature selector.

3. Dropout:

Randomly deactivates a subset of neurons (e.g., 50%) during each training pass. Prevents neurons from co-adapting, acting like an ensemble of 2^N thin sub-networks.

4. Early Stopping:

Monitors validation error during training. Training is terminated the moment validation error starts to rise, preserving optimal generalization.

Essay Question 5

Explain the steps involved in Greedy layerwise training of deep neural network

1. Introduction & Historical Context

Introduced by Geoffrey Hinton (2006), Greedy Layer-Wise Training solved the vanishing gradient barrier in deep networks by training one layer at a time unsupervised (using Restricted Boltzmann Machines or Autoencoders) before global supervised fine-tuning.

2. Step-by-Step Procedure

1. Step 1: Train Layer 1 (Unsupervised)

Feed raw input X to the first hidden layer. Train it as an autoencoder to reconstruct X . Once converged, freeze weights $W^{(1)}$.

2. Step 2: Forward Transform and Train Layer 2

Use the activations of Layer 1 ($h^{(1)} = f(W^{(1)}X)$) as the new input to train Layer 2. Freeze $W^{(2)}$.

3. Step 3: Repeat for All Hidden Layers

Continue training each subsequent layer 1 greedily using the output of layer 1-1.

4. Step 4: Attach Output Layer

Attach a final classification or regression output layer (e.g., Softmax).

5. Step 5: Global Supervised Fine-Tuning

Unfreeze all layers and perform standard end-to-end backpropagation to fine-tune all weights together.

Essay Question 6 (NUMERICAL)

Given input $X = [0.5, 0.6, 0.1]$, weights $W = [0.2, 0.9, 0.3]$, bias $b = 0.5$. Calculate the output using Sigmoid, ReLU, and Tanh activation functions.

Step 1: Computation of Net Input (z)

The net summation equation is: $z = (W \cdot X) + b = w_1x_1 + w_2x_2 + w_3x_3 + b$

$$z = (0.2 \times 0.5) + (0.9 \times 0.6) + (0.3 \times 0.1) + 0.5$$

$$z = 0.10 + 0.54 + 0.03 + 0.50$$

$$z = 1.17$$

Step 2: Applying Activation Functions

(a) Sigmoid Activation Function (σ):

$$\sigma(z) = 1 / (1 + e^{-z}) = 1 / (1 + e^{-1.17})$$

$$e^{-1.17} \approx 0.310363$$

$$\sigma(1.17) = 1 / (1 + 0.310363) = 1 / 1.310363 \approx 0.7631$$

(b) ReLU Activation Function:

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{ReLU}(1.17) = \max(0, 1.17) = 1.17$$

(c) Tanh (Hyperbolic Tangent) Function:

$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$

$$e^{1.17} \approx 3.221993, \quad e^{-1.17} \approx 0.310363$$

$$\tanh(1.17) = (3.221993 - 0.310363) / (3.221993 + 0.310363)$$

$$\tanh(1.17) = 2.91163 / 3.532356 \approx 0.8243$$

Summary Table of Results

Activation Function	Formula	Calculated Output
Sigmoid (σ)	$1 / (1 + e^{-z})$	0.7631
ReLU	$\max(0, z)$	1.1700
Tanh	$(e^z - e^{-z}) / (e^z + e^{-z})$	0.8243

----- LAST-MINUTE EXAM REVISION (UNIT 2) -----

Quick Formula Reference

Concept	Key Formula / Rule
Backprop Chain Rule	$\partial E / \partial w_{ij} = \delta_j \cdot a_i$
Output Error (δ_k)	$\delta_k = (\hat{y}_k - y_k) \cdot f'(z_k)$
Hidden Error (δ_j)	$\delta_j = (\sum \delta_k w_{jk}) \cdot f'(z_j)$
L2 Weight Decay	$J_{\text{reg}} = J_0 + (\lambda/2) \sum w^2$
L1 Sparsity	$J_{\text{reg}} = J_0 + \lambda \sum w $
Sigmoid Formula	$\sigma(z) = 1 / (1 + e^{-z})$
ReLU Formula	$\text{ReLU}(z) = \max(0, z)$

Tanh Formula

$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$